탐색 탭을 Figma 디자인 + 실제 데이터로 바꾸기

이 문서 읽는 법

- 코드 블록은 **직접 쳐요.**
- 📖 설명 / 🤔 왜 / 🏢 현업
- 순서: **STEP 1 (백엔드 한 줄) → STEP 2~4 (프론트) → STEP 5 (화면 확인)**
- STEP 6은 다음에 만들 **공고 적합도** 설계예요(아직 코드 없음).

---

0. 왜 이렇게 하나

```
지금
 ├ Explore.tsx   ← Figma 디자인대로 만든 화면. 그런데 data/mock.ts 샘플 데이터만 씀
 └ LiveData.tsx  ← 실제 API를 쓰지만 디자인 없이 카드만 나열 (← 지금 보이는 "복잡한" 화면)

바꾼 뒤
 └ Explore.tsx   ← Figma 디자인 그대로 + 실제 API 데이터
     └ services/explore.ts  ← API 응답을 화면이 쓰는 모양으로 바꿔 주는 "번역기"
```

- 🤔 **화면을 새로 만들지 않아요.** 이미 디자인대로 만든 화면에 데이터 연결만 바꾸는 게 제일 빠르고 안전해요.
- 🤔 **번역기 파일을 따로 두는 이유:** 서버는 `displayName`, `hireSpecs`처럼 쓰고, 화면은 `name`, `specs`처럼 써요. 변환을 화면 코드 곳곳에 흩어 두면, 서버 응답이 바뀔 때 고칠 곳을 찾기 어려워요. 변환을 한 파일에 모아 둬요.
- 🏢 이 패턴을 **어댑터(Adapter)** 또는 **DTO 매핑**이라고 해요. 서버 응답 모양(DTO)과 화면 모양(도메인 타입)을 분리해요.
- 샘플 데이터는 이제 탐색 화면에서 안 써요. 공고는 크롤링으로 실제 데이터가 들어올 거라서요.

바꾸는 파일

파일	할 일
`server/app/routers/explore.py`	선배 목록에 학과·LinkedIn 열 추가 (한 줄)
`src/types.ts`	`Notice`에 마감일 없음(null), 원문 링크 허용
`src/services/explore.ts`	**새 파일.** API → 화면 데이터 변환
`src/screens/Explore.tsx`	샘플 대신 API 데이터 사용
`src/App.tsx`	탐색 탭을 항상 `Explore`로

STEP 1. 백엔드: 선배 목록에 학과 추가

`server/app/routers/explore.py`의 `seniors` 함수에서 `select(...)` 두 줄을 이렇게 바꿔요.

```
    query = db.table("seniors").select("id, display_name, company, position, career_years, department,
"
                                       "grad_year, job_category, verified, linkedin_url, hire_specs")
```

- 📖 `department`, `linkedin_url` 두 열을 추가했어요.
- 🤔 Figma 선배 카드에는 "회계학과 · 2024년 졸업"과 LinkedIn 아이콘이 있어요. 지금 목록 API는 이 둘을 안 내려줘서 카드가 비어 보여요.
- 🤔 `timeline`(학년별 활동)은 목록에 안 넣어요. 길어서 목록에서는 안 보여 주고, 상세 화면에서만 따로 불러와요. 목록 응답이 가벼워져요.

STEP 2. src/types.ts: 공고 타입 고치기

`Notice` 안의 두 줄을 바꿔요.

```
  dday: number;
  tags: string[];
```

→

```
  dday: number | null;
  tags: string[];
  url?: string | null;
```

- 📖 `number | null`: 마감일이 없는 공고도 있어요(학교 공지 중 상당수). 그럴 땐 D-day 배지를 안 그려요.
- 📖 `url?`: 원문 보기 링크예요. `?`는 "없어도 된다"는 뜻이에요.

STEP 3. 새 파일 src/services/explore.ts

3-① import와 서버 응답 타입

```
import { backend } from './backend';
import type { Notice, SchoolStats, Senior } from '../types';

type PostingDto = { id: string; type: 'CAMPUS' | 'EXTERNAL'; tag: string | null; title: string; organi
zation: string | null; description: string; dDay: number | null; url: string | null };
type StatDto = { year: number; label: string; value: number; unit: string; source: string | null };
type SeniorDto = {
  id: string; displayName: string; company: string; position: string; careerYears: number | null;
  department: string | null; gradYear: number | null; jobCategory: string | null; verified: boolean;
  linkedinUrl?: string | null; hireSpecs: string[]; timeline?: { grade: string; text: string }[];
};
```

- 📖 `Dto`(Data Transfer Object): **서버가 보내 주는 모양 그대로** 적은 타입이에요. 백엔드의 `camel_keys`가 `display_name` → `displayName`으로 바꿔서 보내요.
- 📖 `backend()`: 이미 있는 함수예요. 로그인 토큰을 붙이고, 토큰이 만료되면 자동으로 다시 받아요.

3-② 공고

```
const toNotice = (p: PostingDto): Notice => ({
  id: p.id,
  type: p.tag ?? (p.type === 'CAMPUS' ? '교내' : '교외'),
  title: p.title,
  desc: [p.organization, p.description].filter(Boolean).join(' · '),
  dday: p.dDay,
  tags: p.tag ? [p.tag] : [],
  url: p.url,
});

export async function loadNotices(): Promise<Notice[]> {
  return (await backend<{ items: PostingDto[] }>('/explore/postings')).items.map(toNotice);
}
```

- 📖 `p.tag ?? ...`: 태그(특강, 비교과…)가 있으면 그걸 배지에 쓰고, 없으면 "교내"나 "교외"로 표시해요. `??`는 "앞이 null이면 뒤를 써라"예요.
- 📖 `[a, b].filter(Boolean).join(' · ')`: 둘 중 비어 있는 건 빼고 " · "로 이어요. 주최 기관이 없어도 "null · 설명"처럼 나오지 않아요.
- 🤔 마감 지난 공고는 서버(`/explore/postings`)가 이미 걸러서 보내요. 프론트에서 또 거를 필요 없어요.

3-③ 학교 취업 통계

```
const fmt = (s: StatDto) => `${s.value.toLocaleString()}${s.unit}`;

export async function loadSchoolStats(): Promise<SchoolStats> {
  const [school, byJob] = await Promise.all([
    backend<{ items: StatDto[] }>('/explore/employment-stats'),
    backend<{ items: StatDto[] }>('/explore/employment-stats/by-job'),
  ]);
  const year = Math.max(...school.items.map(s => s.year), ...byJob.items.map(s => s.year));
  const pick = (label: string) => school.items.find(s => s.year === year && s.label === label);
  const shown = [['취업률', '취업률'], ['졸업자 수', '졸업자'], ['4대보험 직장취업자', '직장 취업']] as
const;
  return {
    year: String(year),
    source: pick('취업률')?.source ?? '수원대학교 취업통계 공시',
    updateCycle: '매년 공시 후 갱신',
    overall: shown.map(([label, title]) => { const s = pick(label); return { label: title, value: s ?
fmt(s) : null }; }),
    byJob: byJob.items.filter(s => s.year === year).slice(0, 6).map(s => ({ label: s.label, value: s.v
alue })),
  };
}
```

- 📖 `Promise.all([...])`: API 두 개를 **동시에** 불러요. 하나씩 기다리는 것보다 빨라요.
- 📖 `year = Math.max(...)`: 가장 최근 연도(2025)만 보여 줘요. 2023, 2024 취업률이 한 화면에 섞이던 문제가 없어져요.
- 📖 `shown`: Figma의 숫자 칸 3개(취업률 / 졸업자 / 직장 취업)에 넣을 통계를 **이름으로 골라요.** 왼쪽은 DB 라벨, 오른쪽은 화면에 쓸 짧은 이름이에요.
- 📖 `.slice(0, 6)`: 직무별 진출 현황은 상위 6개만 보여 줘요. 서버가 이미 큰 순서로 보내요.
- 📖 `toLocaleString()`: 1902 → "1,902"처럼 쉼표를 찍어요.
- 😥 값이 없으면 `null`로 둬요. 화면은 이미 `null`이면 "자료 준비 중"으로 보여 주게 만들어져 있어요.

3-④ 선배

```
const toSenior = (s: SeniorDto): Senior => ({
  id: s.id,
  name: s.displayName,
  initial: s.displayName.slice(0, 1),
  company: s.company,
  role: s.position,
  position: s.careerYears ? `경력 ${s.careerYears}년차` : '경력 정보 없음',
  industry: s.jobCategory ?? '기타',
  dept: s.department ?? '학과 미공개',
  classOf: s.gradYear ? `${s.gradYear}년 졸업` : '',
  verified: s.verified,
  specs: s.hireSpecs,
  links: { linkedin: s.linkedinUrl ?? '', github: '' },
  timeline: (s.timeline ?? []).map(t => ({ when: t.grade, what: t.text })),
});

export async function loadSeniors(): Promise<Senior[]> {
  return (await backend<{ items: SeniorDto[] }>('/explore/seniors')).items.map(toSenior);
}

export async function loadSenior(id: string): Promise<Senior> {
  return toSenior(await backend<SeniorDto>(`/explore/seniors/${encodeURIComponent(id)}`));
}

export async function askSenior(id: string, content: string) {
  await backend(`/explore/seniors/${encodeURIComponent(id)}/questions`, { method: 'POST', body: JSON.s
tringify({ content }) });
}
```

- 📖 `initial` : 아바타 동그라미에 들어갈 첫 글자예요("김OO" → "김").
- 📖 `industry: s.jobCategory` : Figma의 선배 필터 칩(전체 / IT 플랫폼 / 공기업…)이 이 값으로 만들어져요.
- 📖 `timeline ?? []` : 목록 응답엔 timeline이 없어서 빈 배열로 둬요. 상세를 불러오면 채워져요.
- 📖 `encodeURIComponent(id)` : 주소에 넣을 값은 항상 감싸요. 이상한 글자가 들어와도 주소가 깨지지 않아요.

STEP 4. src/screens/Explore.tsx

4-① import 바꾸기

맨 위의 이 세 줄을

```
import { demoMode } from '../services/http';
import { useState } from 'react';
...
import { notices, schoolStats, seniors } from '../data/mock';
```

이렇게 바꿔요. 첫 줄(`demoMode`)은 **지우고**, 나머지 두 줄은 아래로 바꿔요.

```
import { useEffect, useState } from 'react';
...
import { askSenior, loadNotices, loadSchoolStats, loadSenior, loadSeniors } from '../services/explor
e';
```

4-② 불러오기 공통 함수: import 바로 아래(type ExploreTab 위)에 추가

```
function useLoad<T>(load: () => Promise<T>) {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState('');
  const [retry, setRetry] = useState(0);
  useEffect(() => {
    let active = true;
    setError('');
    load().then(v => { if (active) setData(v); }).catch(e => { if (active) setError(e instanceof Error
? e.message : '불러오지 못했어요.'); });
    return () => { active = false; };
  }, [retry]);
  const status = error
    ? <div className="notice notice--point" role="alert">{error} <button className="link" onClick={()
=> setRetry(n => n + 1)}>다시 시도</button></div>
    : data === null ? <p className="t-caption" role="status">불러오는 중</p> : null;
  return { data, status };
}
```

- 📖 이름이 `use`로 시작하는 함수는 **React 훅**이에요. 화면 세 곳(공고, 통계, 선배)이 똑같이 "불러오기 → 로딩 → 에러 → 다시 시도"를 하니까 한 번만 만들어 재사용해요.
- 📖 `active` 변수: 불러오는 도중에 다른 탭으로 넘어가면, 늦게 도착한 응답이 사라진 화면을 건드리지 않게 막아요.
- 📖 `retry`: "다시 시도"를 누르면 숫자가 1 올라가고, 그걸 감지해서 `useEffect`가 다시 불러와요.

4-③ 샘플 안내 문구 지우기

`Explore` 함수 안의 이 줄을 **지워요.**

```
      {!demoMode && <p className="notice notice--point" role="status">이 화면은 아직 샘플 데이터입니다.
서버 연동 전입니다.</p>}
```

그리고 `if (senior) return <SeniorDetail s={senior} ...` 줄의 `s={senior}`를 `summary={senior}`로 바꿔요.

```
  if (senior) return <SeniorDetail summary={senior} onBack={() => setSenior(null)} toast={toast} />;
```

4-④ 공고 목록 NoticeList

함수 첫 부분을

```
function NoticeList({ goTab }: { goTab: (tab: TabId) => void }) {
  return (
    <div className="stack-8">
      {notices.map((n) => (
```

→

```
function NoticeList({ goTab }: { goTab: (tab: TabId) => void }) {
  const { data: notices, status } = useLoad(loadNotices);
  if (!notices) return status;
  return (
    <div className="stack-8">
      {status}
      {notices.length === 0 && <p className="t-caption">지금 모집 중인 공고가 없어요.</p>}
      {notices.map((n) => (
```

D-day 배지 줄을

```
            <span className={`badge ${n.dday <= 7 ? 'badge--point' : 'badge--gray'}`}>D-{n.dday} 마감
</span>
```

→

```
            {n.dday !== null && <span className={`badge ${n.dday <= 7 ? 'badge--point' : 'badge--gra
y'}`}>{n.dday === 0 ? '오늘 마감' : `D-${n.dday} 마감`}</span>}
```

`<div className="t-caption">{n.desc}</div>` 바로 아래에 원문 링크 한 줄:

```
          {n.url && /^https?:\/\//i.test(n.url) && <a className="link" href={n.url} target="_blank" re
l="noreferrer">원문 보기 <Icon name="external" size={12} /></a>}
```

- 📖 `if (!notices) return status;` : 아직 데이터가 없으면 "불러오는 중"이나 에러만 보여 줘요.
- 📖 `/^https?:\/\//i.test(...)` : `http://` 나 `https://` 로 시작하는 주소만 링크로 만들어요. `javascript:` 같은 위험한 주소가 DB에 들어와도 실행되지 않아요.
- 📖 `rel="noreferrer"` : 새 탭으로 연 학교 페이지가 우리 앱 창을 건드리지 못하게 해요.

4-⑤ 통계 `SchoolStatsView`

```
function SchoolStatsView() {
  const max = Math.max(...schoolStats.byJob.map((r) => r.value));
```

→

```
function SchoolStatsView() {
  const { data: schoolStats, status } = useLoad(loadSchoolStats);
  if (!schoolStats) return status;
  const max = Math.max(1, ...schoolStats.byJob.map((r) => r.value));
```

- 📖 `Math.max(1, ...)` : 데이터가 비었을 때 0으로 나누는 걸 막아요(막대 너비 계산).
- 나머지 화면 코드는 그대로예요. 변수 이름(`schoolStats`)을 맞춰 둬서 아래 JSX를 안 고쳐도 돼요.

4-⑥ 선배 목록 SeniorList

```
function SeniorList({ onOpen }: { onOpen: (s: Senior) => void }) {
  const industries = ['전체', ...new Set(seniors.map((s) => s.industry))];
  const [ind, setInd] = useState('전체');
```

→

```
function SeniorList({ onOpen }: { onOpen: (s: Senior) => void }) {
  const { data: seniors, status } = useLoad(loadSeniors);
  const [ind, setInd] = useState('전체');
  if (!seniors) return status;
  const industries = ['전체', ...new Set(seniors.map((s) => s.industry))];
```

- 🧐 **순서가 중요해요.** `useState` 는 `if (...) return` **위에** 있어야 해요. React 훅은 매번 같은 순서로 불려야 하는데, return 아래에 두면 로딩 중일 때는 안 불리고 다 불러온 뒤에만 불려서 에러가 나요.

4-⑦ 선배 상세 SeniorDetail

함수 첫 부분을

```
function SeniorDetail({ s, onBack, toast }: { s: Senior; onBack: () => void; toast: Toast }) {
  const [ask, setAsk] = useState(false);
  const [q, setQ] = useState('');
```

→

```
function SeniorDetail({ summary, onBack, toast }: { summary: Senior; onBack: () => void; toast: Toast
}) {
  const { data: detail } = useLoad(() => loadSenior(summary.id));
  const s = detail ?? summary;
  const [ask, setAsk] = useState(false);
  const [q, setQ] = useState('');
  const [busy, setBusy] = useState(false);
  async function submit() {
    setBusy(true);
    try { await askSenior(s.id, q.trim()); setAsk(false); setQ(''); toast('질문을 등록했어요'); }
    catch (e) { toast(e instanceof Error ? e.message : '질문을 등록하지 못했어요.'); }
    finally { setBusy(false); }
  }
```

맨 아래 "질문 등록" 버튼 줄을

```
          <button className="btn btn--primary btn--block" disabled={!q.trim()} onClick={() => { setAsk
(false); setQ(''); toast('질문을 등록했어요'); }}>질문 등록</button>
```

→

```
          <button className="btn btn--primary btn--block" disabled={!q.trim() || busy} onClick={() =>
void submit()}>{busy ? '등록 중' : '질문 등록'}</button>
```

- 📖 `s = detail ?? summary`: 목록에서 받은 정보(`summary`)로 **바로 화면을 그리고**, 상세(타임라인 포함)가 도착하면 바꿔 끼워요. 누르자마자 빈 화면이 뜨지 않아요.
- 🏬 이걸 **낙관적 표시(optimistic UI)** 또는 **placeholder data**라고 해요. 이미 아는 정보로 먼저 보여주고 나머지를 채워요.
- 📖 예전 버튼은 서버에 안 보내고 "등록했어요"만 띄웠어요(가짜). 이제 진짜로 `POST /explore/seniors/{id}/questions`를 불러요.

STEP 4-⑧. src/App.tsx

import 줄에서 `LiveExplore`를 빼요.

```
import { LiveMindmap, LiveSettings } from './screens/LiveData';
```

`screens` 안의 `explore:` 줄을

```
    explore: <Explore profile={profile} goTab={setTab} toast={toast} />,
```

- 📖 이제 탐색 탭은 항상 디자인된 `Explore` 화면이에요. `LiveData.tsx` 안의 `LiveExplore` 함수는 안 쓰게 돼요(나중에 지워도 돼요).

STEP 5. 확인

1. B 서버(8001)와 프론트를 켜요.
2. `npm run typecheck` → 에러 없이 끝나야 해요.
3. 탐색 탭에서 확인해요.

탭	보여야 하는 것
교내외 공고	태그 배지, D-day 배지(마감 7일 이내는 강조색), 제목, 주최·설명, 원문 보기, "이 공고로 맞춤 서류 만들기"
학교 취업 통계	2025 숫자 칸 3개(63.4% / 1,902명 / 944명), 직무별 진출 막대 6개, 출처 안내
선배 Q&A	직무 칩 필터, 선배 카드(아바타, 회사, 직무 · 경력, 학과 · 졸업연도, 스펙 3개 + 더보기)
선배 상세	스펙 전체, 학년별 타임라인, 질문 남기기 → Supabase `senior_questions`에 한 줄 생김

> 공고 탭이 비어 있으면 정상일 수 있어요. 지금 DB 공고는 마감일이 11월이라 하나둘 빠지고 있어요.

> 크롤링(다음 작업)이 들어오면 채워져요.

이번 범위 밖 (알아 두기)

- **홈 화면의 추천 공고 카드**(`Home.tsx`)도 아직 `mock.ts`의 샘플을 써요. 크롤링 작업 때 같이 바꿔요.
- 역량 마인드맵(`Mindmap.tsx`) 디자인 화면도 같은 방식(샘플 → 실제 API)으로 바꿀 수 있어요.

STEP 6. (설계) 공고마다 "내 직무·목표 기업과 얼마나 맞는지"

의견: 좋은 기능이에요. 다만 이렇게 하는 걸 추천해요

정할 것	추천	이유
표시 방법	**3단계 배지** (`직무 관련 높음 / 보통`, 낮으면 표시 안 함) + 이유 한 줄	"73% 적합"처럼 숫자를 보여 주면 정밀해 보이지만 근거가 약해요. 학생이 숫자를 믿고 판단하면 오히려 해가 돼요
직무 적합도	**항상 계산**	특강, 비교과, 채용 모두 직무와 관련이 있어서 의미가 있어요
목표 기업 적합도	**관련 있을 때만 배지** (예: "목표 기업 삼성전자 채용설명회", "목표 기업과 같은 반도체 업계")	교내 공지 대부분(자소서 특강, 박람회 안내)은 특정 기업과 상관없어요. 매번 "목표 기업 적합도 낮음"이 뜨면 화면만 시끄러워져요
계산 시점	**공고를 수집할 때 AI가 한 번 태그를 붙여 두고**, 화면을 열 때는 태그끼리 비교만	사용자가 탭을 열 때마다 AI를 부르면 느리고, Gemini 무료 한도도 금방 차요
정렬	관련 높음 → 마감 임박 순	나한테 맞는 게 위로 와요

흐름

```
[수집할 때, 공고마다 1번]  Gemini → { jobTags: ["데이터 분석", "마케팅"], companies: ["삼성전자"],
                                      industries: ["반도체"], reasonHint: "SQL·데이터 시각화 실습" }
                                   → explore_postings에 저장

[화면 열 때, AI 없이]  GET /explore/postings
   내 희망 직무(desired_job) ⇔ jobTags     → 높음 / 보통 / 없음
   내 목표 기업(target_company) ⇔ companies → "목표 기업 공고"
                                ⇔ industries → "목표 기업과 같은 업계"
   → 각 공고에 { jobFit: "HIGH", jobFitReason: "...", companyFit: "SAME_COMPANY" | "SAME_INDUSTRY" | null }
```

- 직무 이름은 사람마다 다르게 써요("데이터 분석가" / "데이터 애널리스트"). 그래서 희망 직무도 저장할 때 **같은 직무 분류표**(예: 직무 12개)로 한 번 맞춰 두고 비교해요.
- 이 기능은 **공고 크롤링(태그 붙이기)이 먼저** 있어야 해요. 그래서 크롤링 가이드 다음 단계로 붙여요.